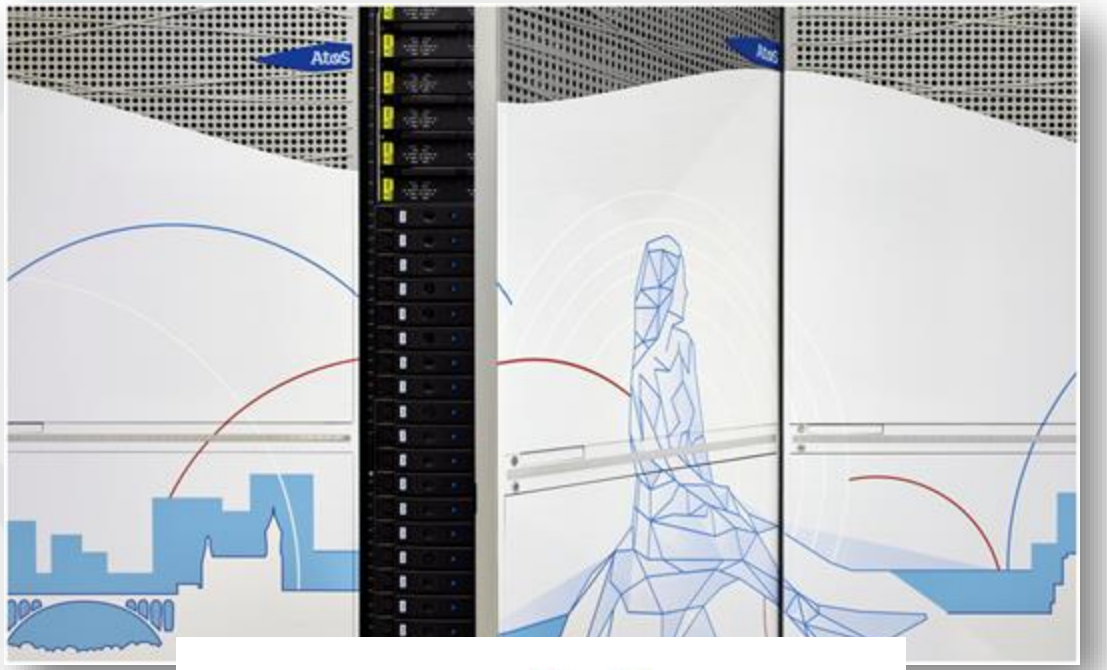


MeluXina Onboarding Workshop

LUX PROVIDE

Agenda

1. **Welcome & Introduction**
2. **MeluXina:**
    - Architecture
    - Getting Access
    - Data transfers
    - Job Submission
    - Monitoring your jobs
    - Software Stack
    - Containerization
3. **Questions & Discussion**

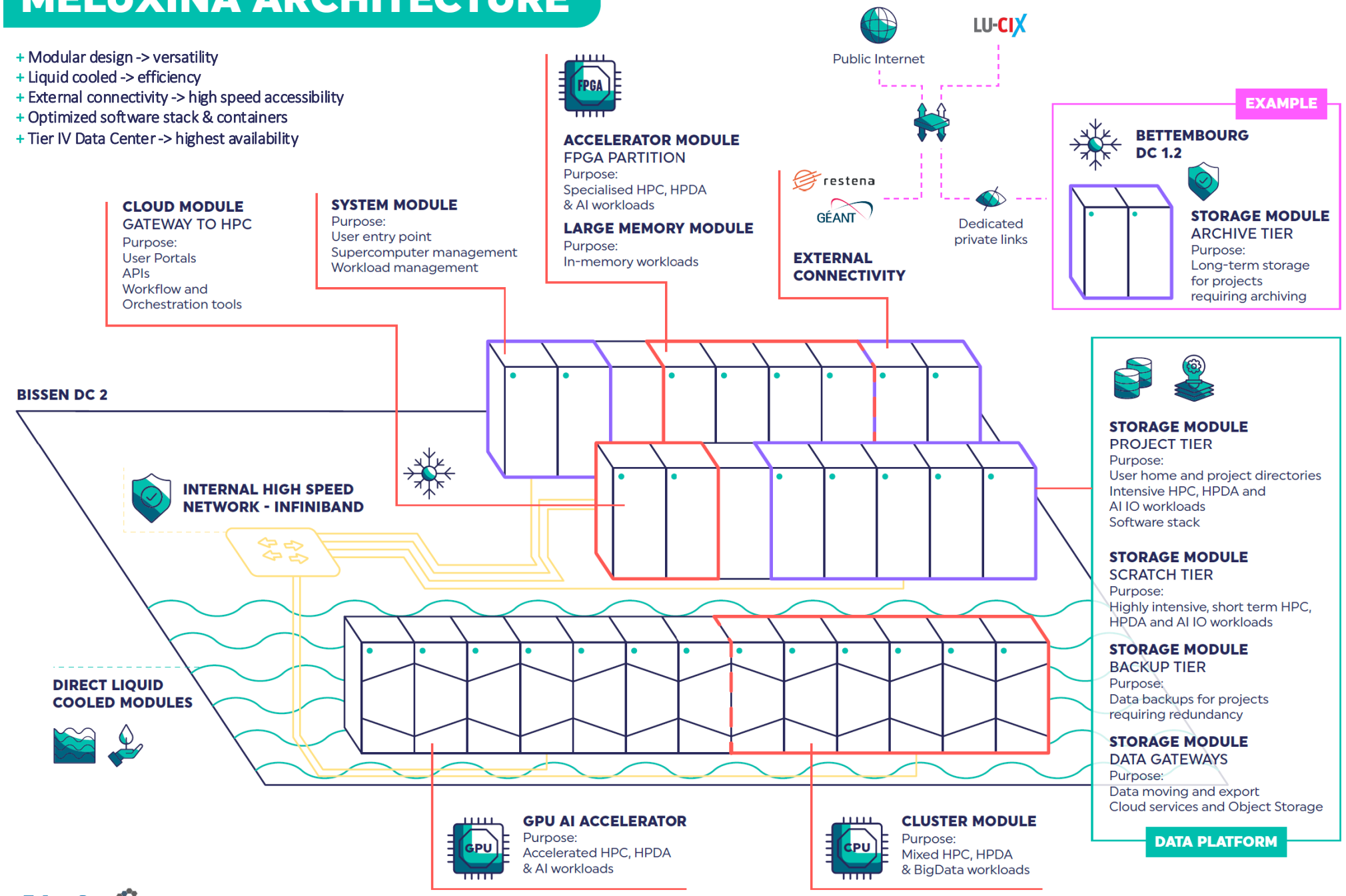


MELUXINA

MeluXina: Architecture

MELUXINA ARCHITECTURE

+ Modular design -> versatility
+ Liquid cooled -> efficiency
+ External connectivity -> high speed accessibility
+ Optimized software stack & containers
+ Tier IV Data Center -> highest availability

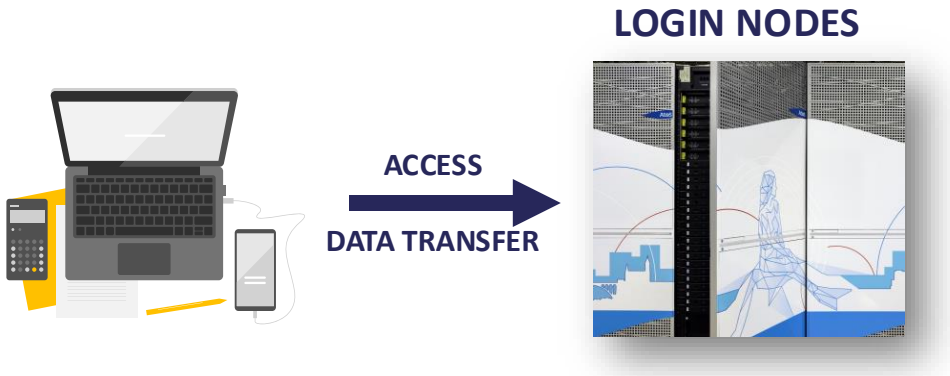


MeluXina: Getting Access

Getting Access: *Login nodes*

Linux / macOS / WSL: SSH Connection

ssh '<your-user-ID>'@login.lxp.lu -p 8822 -i ~/.ssh/id_ed25519_mlux



- **4 login nodes** in total *(Load-Balanced)*
- Can switch between login nodes once connected

Host meluxina
User <your-user-ID>
Port 8822
Hostname login.lxp.lu
IdentityFile ~/.ssh/id_ed25519_mlux
ConnectTimeout 15
ServerAliveCountMax 30
ServerAliveInterval 60
ForwardAgent yes

Requirements: **Ed25519 key pair-based authentication***

Windows: MobaXterm

https://mobaxterm.mobatek.net/

***Steps to generate appropriate key pairs:**
https://docs.lxp.lu/first-steps/connecting/

Getting Access: *workflow*



ACCESS

DATA TRANSFER

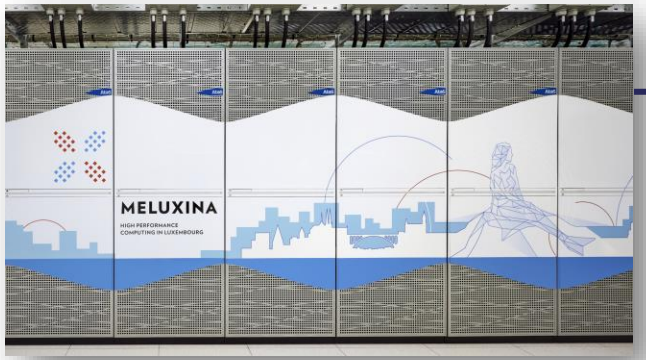
LOGIN NODES

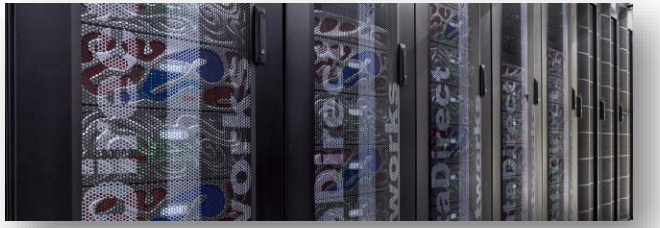


TASK

SUBMISSION

COMPUTE NODES





DATA TIERS

Login nodes:

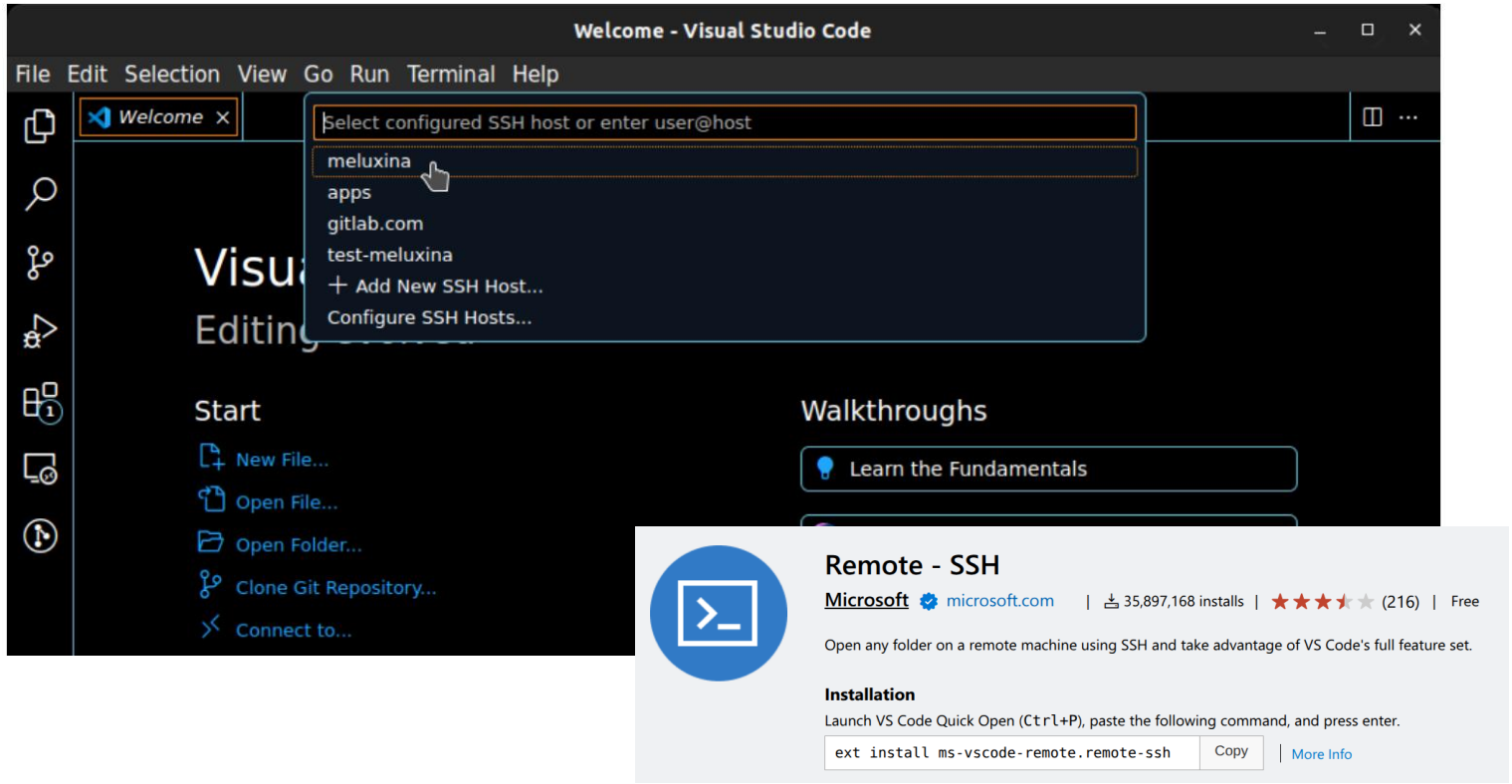
Intended for job submission, non-intensive interactive tasks, file management, and software development - **NOT for computation, No module available**

Compute nodes:

- **Access to software environment.**
- **Designated for pipeline preparation & computational workload execution.**

Getting Access: *high level access & development*

VS Code: a powerful and convenient way to streamline remote development

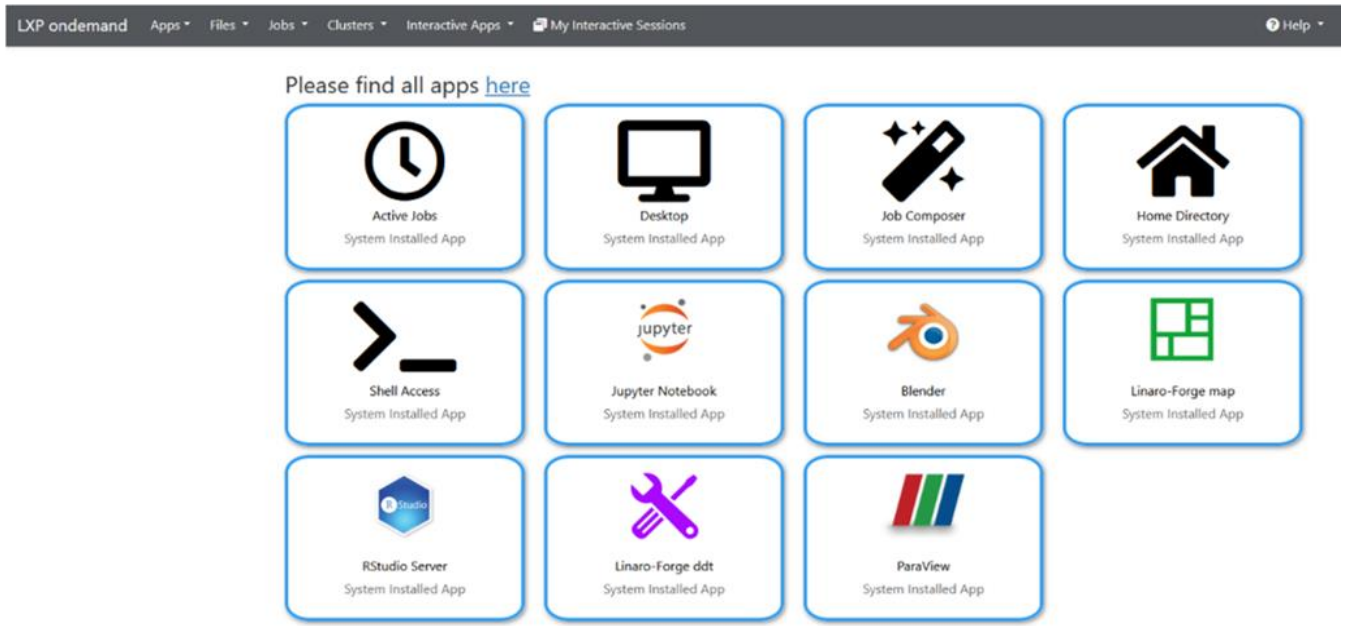


- Use VS Code's Remote - SSH extension to **securely connect and browse HPC files as if local**
- **Edit** code, scripts & Jupyter notebooks **within the VS Code interface**
- **Run** computations and notebooks **directly on the HPC**

Guide on installing & using VS Code on MeluXina:
https://docs.lxp.lu/howto/VSCodeOnMeluxina/

Getting Access: *OpenOnDemand*





- Access **OpenOnDemand** at https://portal.lxp.lu/
- Authentication is managed via **Keycloak (MFA)**: https://docs.lxp.lu/web_services/keycloak/
- Run **GUI** applications on MeluXina
- OpenOnDemand also provides a **shell access**

Guide on launching JupyterLab:
https://docs.lxp.lu/web_services/open_ondemand/howtojlab/

MeluXina: Data Transfers

Data transfers: *Storage paths*

Data allocations:

- Users: **private Home directory** /home/users/u000000
  and **access to Project directories** they're part of

- Projects:

a **Home directory** /project/home/p000000
a **SCRATCH directory** /project/scratch/p000000

Tier1: HOME

- Permanent storage
- Usually Small quota
- Use for scripts, configs

Tier2: SCRATCH

- Large, fast storage
- Temporary files, datasets, job outputs
- Not backed up

Tier0: LOCAL

- Local compute node storage
- Buffer or cache for big workflows
- Flushed at end of reservation



Guide on MeluXina storage:
https://docs.lxp.lu/first-steps/managing_data/

Data transfers

scp (command-line)

- Transfer a full directory from your machine, add the option –r (recursive)

scp –r –P 8822 -i ~/.ssh/id_ed25519_mlux folder/ **'<your-user-ID>'@login.lxp.lu:/remote/path**

(with ssh config): **scp -r** folder/ **meluxina:/remote/path**

- Transfer a full directory back to your machine, just invert folder/ and **/remote/path**

rsync (command-line)

- Transfer a full directory from your machine

rsync -az -e 'ssh -p 2222' -i ~/.ssh/id_ed25519_mlux file.txt **'<your-user-ID>'@login.lxp.lu:/remote/path**

(with ssh config): **rsync -az** folder/ **meluxina:/remote/path**

- Transfer a full directory back to your machine, just invert folder/ and **/remote/path**
- **Parameters:**
  - - a is for archive mode
  - - v is for verbose mode
  - - z is for compressed mode



ACCESS

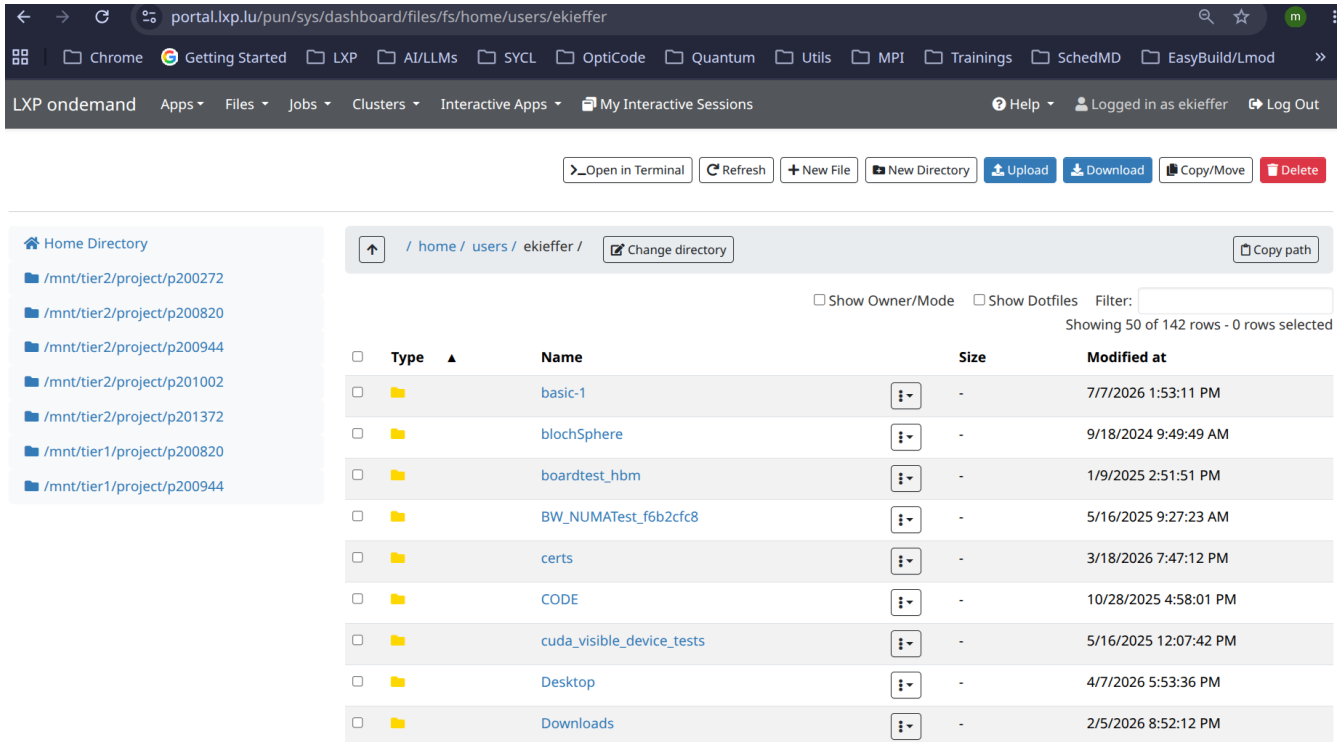
DATA TRANSFER

LOGIN NODES

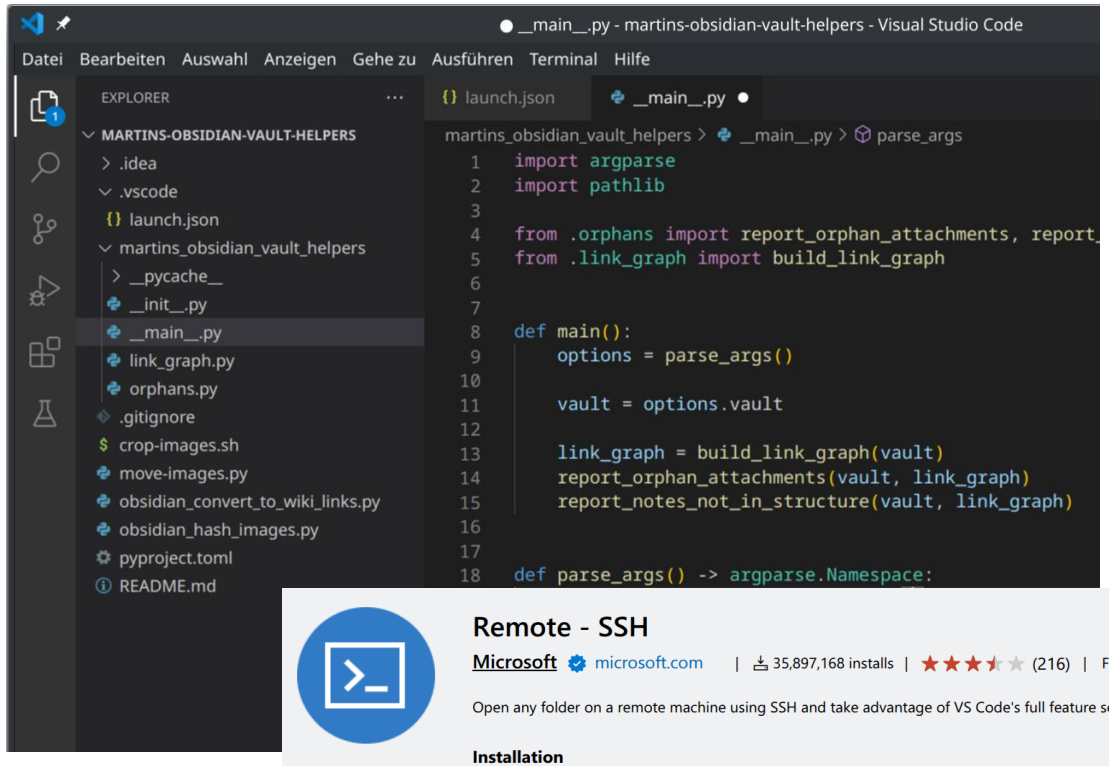


Data transfers: *GUI*

OpenOnDemand



VS Code





Data transfer guide:
https://docs.lxp.lu/first-steps/transferring_data/

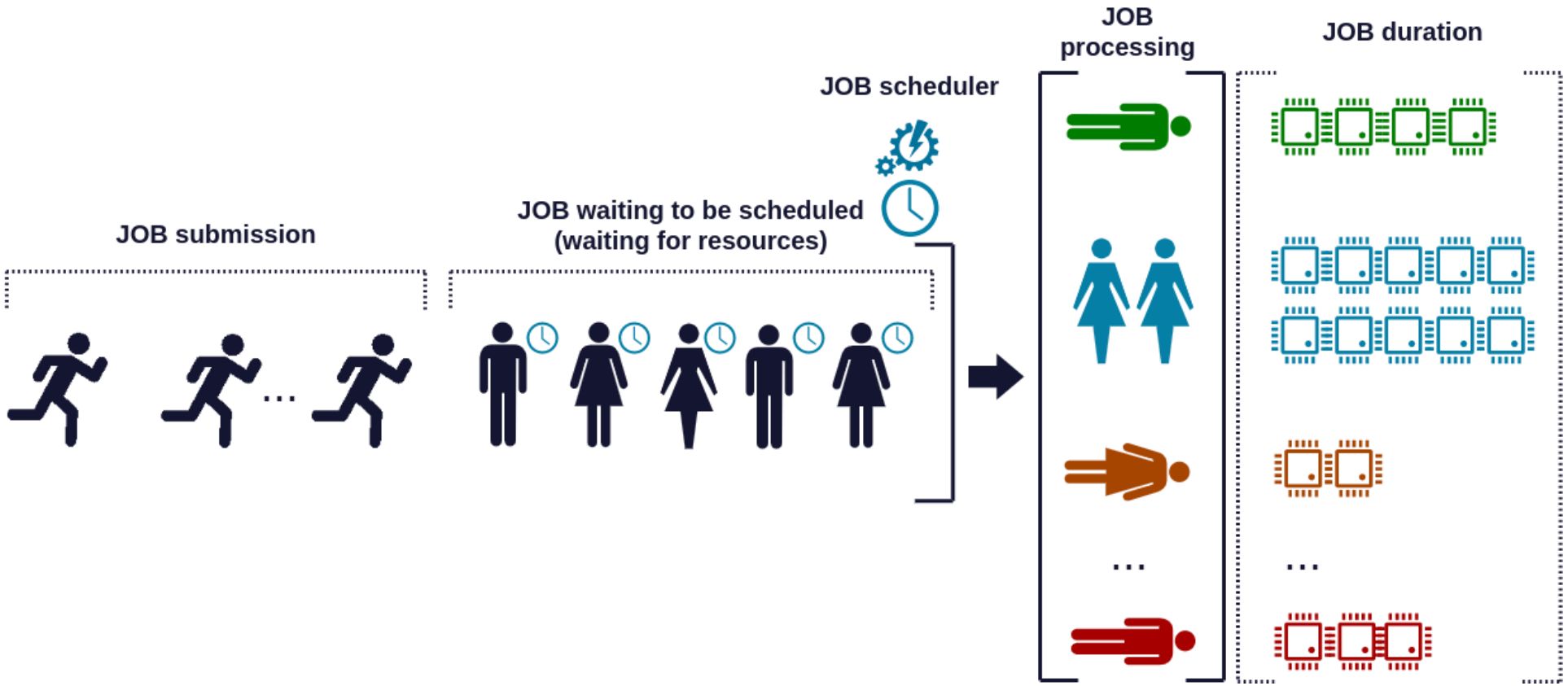
MeluXina: SLURM

Our job scheduler

Handling jobs:

On a supercomputer, **user applications are run through a job scheduler**, also called **a batch scheduler** or queueing system.

MeluXina is using the **SLURM Workload Manager** as job scheduler.



Handling jobs: *Interactive session*

Choose Interactive sessions *for exploration, debugging & small tests*

Allocation command:

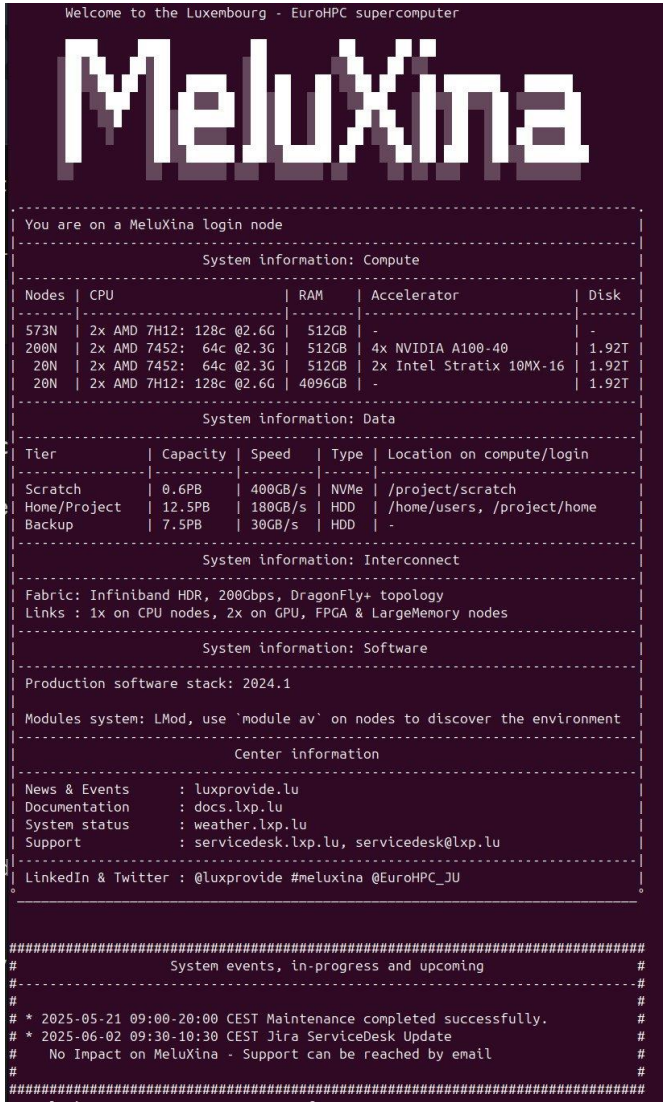
$ **salloc** -A p000000 **-p cpu** --qos default **-N 1** **-t 0:30:0**

-A	Account: charges job to specified account	*mandatory*
-p	Partition requested	*mandatory*
--qos	Quality of Service	*mandatory*
-N	Number of nodes on which to run	*optional*
-t	Time limit in *hours:minutes:seconds* format	*mandatory*

This example will allocate ***1 full CPU node*** *in interactive mode for* ***30 minutes***.



https://slurm.schedmd.com/salloc.html



Handling jobs: *Submit a Batch job*

Submit a Batch job *for production runs, heavy and long computations.*

Allocation command:

$ **sbatch mylauncher.sh**

This file is split in two important parts:

1. The **top half of the file** includes a set of **#SBATCH options** which are **meta-commands to the SLURM scheduler,** instructing SLURM on your resource requirements.

2. After the #SBATCH options section, **user instructions are in the second section.**

Upon submission, you can **overwrite #SBATCH options**:

$ **sbatch --qos=test mylauncher.sh**

1.
```
#!/bin/bash -l

#SBATCH --account=p000000
#SBATCH --partition=cpu
#SBATCH --qos=short
#SBATCH --time=0:5:0
#SBATCH --nodes=2
#SBATCH --ntasks-per-node=2
```

2.
```
srun -l hostname
```

This example requests ***two CPU nodes, for 5 minutes***

Handling jobs: *Quality of Service (QOS)*

It's a SLURM scheduling feature that lets users choose between different resource usage profiles

$ **salloc** -A p000000 -p cpu **--qos default** -t 00:30:00

QOS	Max. Time (hh:mm)	Max. nodes per job	Priority	Used for..
dev	06:00	1	Regular	Interactive executions for code/workflow development, with a maximum of 1 job per user; QOS linked to special reservations
test	00:30	5%	High	Testing and debugging, with a maximum of 1 job per user
short	06:00	5%	Regular	Small jobs for backfilling
short-preempt	06:00	5%	Regular	Small jobs for backfilling
default	**48:00**	**25%**	**Regular**	**Standard QOS for production jobs**
long	144:00	5%	Low	Non-scalable executions with a maximum of 1 job per user
large	24:00	70%	Regular	Very large-scale executions by special arrangement, max 1 job per user
urgent	06:00	5%	Very high	Urgent computing needs, by special arrangement

MeluXina: Monitoring your jobs

Monitoring resources: *myquota*

$ myquota *available on the MeluXina login nodes and compute nodes*

Provides a detailed summary of your **compute and storage allocations per project.**

Compute time:

- Tracked in Node-Hours, separately for each resource type (CPU, GPU, FPGA, LargeMem)
- Shows used vs. total available compute hours for the current month

Data storage:

- Reports storage usage in GiB (for data size) and in number of files (for the metadata)
- Shows current usage out of maximum allocation per project

```
                                COMPUTE ALLOCATIONS FOR CURRENT MONTH
                                COMPUTE USAGE FROM  2026-08-01 to NOW
Project                  CPU node-hours            GPU node-hours            FPGA node-hours          LargeMem node-hours
 p000000                 0                    1812   9498   19.1%             0                         0
                                          DATA ALLOCATIONS
 Datapath                          Data GiB                          No. Files
/home/users/u000000          80 |     100 |  80%           84458 |     100000 |  84%
/project/scratch/p000000    110 |    5000 |   2%             258 |    1000000 |   0%
/project/home/p000000      1801 |    5000 |  36%           71189 |    1000000 |   7%
```

Monitoring jobs

$ squeue

- **displays the list of jobs** currently in the SLURM queue (pending or running).
- *available on the MeluXina login nodes and compute nodes*

```
  JOBID PARTITION     NAME     USER    ACCOUNT    STATE       TIME   TIME_LIMIT  NODES NODELIST(REASON)
5147364       gpu Qwen3.5-  u000000    p000000 CONFIGUR       0:06      1:00:00      2 mel[2017,2146]
```

To monitor status of a **specific running job**, you can **connect to the particular node** with the following command: **$ ssh melxxxx** *(mel2017 with the example above)*

Once you are connected to the node allocated, you can use **'htop'** or **'nvidia-smi'** (GPU nodes)

Monitoring jobs

$ scontrol

- **scontrol show job <jobid> :** Retrieve deep configuration specs of a running or pending job

- **scontrol update jobid=<jobid> qos=<qos>:** Change a pending job's parameters.

After a job completed, you can query it with **scontrol** for 5 min. Afterwards, you will need **sacct**.

$ sacct

- shows **detailed information** and history of finished or running jobs from SlurmDB

sacct -T -S2014-07-03-11:40 -E2014-07-03-12:00 -X -ojobid,jobname,user,start,end,state

JobID	Start	End	State
2	2014-07-03T11:40:00	2014-07-03T11:59:01	COMPLETED
3	2014-07-03T11:40:00	2014-07-03T12:00:00	RUNNING
4	2014-07-03T11:40:00	2014-07-03T11:45:21	COMPLETED
5	2014-07-03T11:41:01	2014-07-03T12:00:00	RUNNING

$ scancel

- **cancels a running or pending job** using its Job ID

MeluXina: Software Stack

MeluXina User Software Stack (MUSE)

Software stack: *pre-optimized packages*

- Access to hundreds of **pre-installed scientific & engineering packages** (e.g., Python, scikit-learn, PyTorch, Seaborn, TensorFlow, GROMACS, compilers, JAVA)
- The **Software is pre-compiled and optimized** for our HPC hardware — saving time and boosting performance.
- Available via an easy-to-use **module system** — no need to install locally:
  - New release every year → env/release/2025.1 (current default) –2025.1 based on toolchains 2025a
  - Updated during the year→ env/staging/2025.1 – Additions and modifications to the release – Possibly many staging environments per year
  - Possibly local environments for project

```
------------------------------------------ /apps/USE/system/modules ------------------------------------------
   env/release/2021.3       env/release/2023.1 (S)       env/staging/2021.5    env/staging/2024.1             S:  Module is Sticky
   env/release/2021.5       env/release/2024.1 (S)       env/staging/2022.1    env/staging/2025.1 (D)         L:  Module is loaded
   env/release/2022.1 (S)   env/release/2025.1 (S,L,D)   env/staging/2023.1                                   D:  Default Module
```

Software stack: *environment modules*

$ module The **core command** by which software modules can be listed, loaded (activated), and unloaded, followed by an appropriate command.

Module Command	Description
module avail	List/browse available modules
module list	Show modules currently loaded (active)
module load module_name	Load a specific module (may load additional modules as dependencies)
module unload module_name	Unload a loaded module (does not unload modules activated as dependencies)
module purge	Unload all loaded modules
module spider	List all modules and the short description of each

MeluXina uses the **Lmod software modules system**

The **module command is only available on compute nodes** and not on the login nodes.

The **applications** provided through the software modules system **must be used only on compute nodes.**

MeluXina: Containerization

Container system : *Apptainer*

On MeluXina the **Apptainer container system** is provided to support containerized workloads.

- Docker & OCI compatible
- Non-privileged mode for improved security
- Support for GPU accelerated applications
- Support for creating and running encrypted containers
- Support for trusted containers: PGP signed & verified
- Compressed read-only Singularity Image File (SIF) format, suitable for production (default)
- Writable (ch)root directory called a sandbox, for interactive development (--sandbox option)

How to run containers?

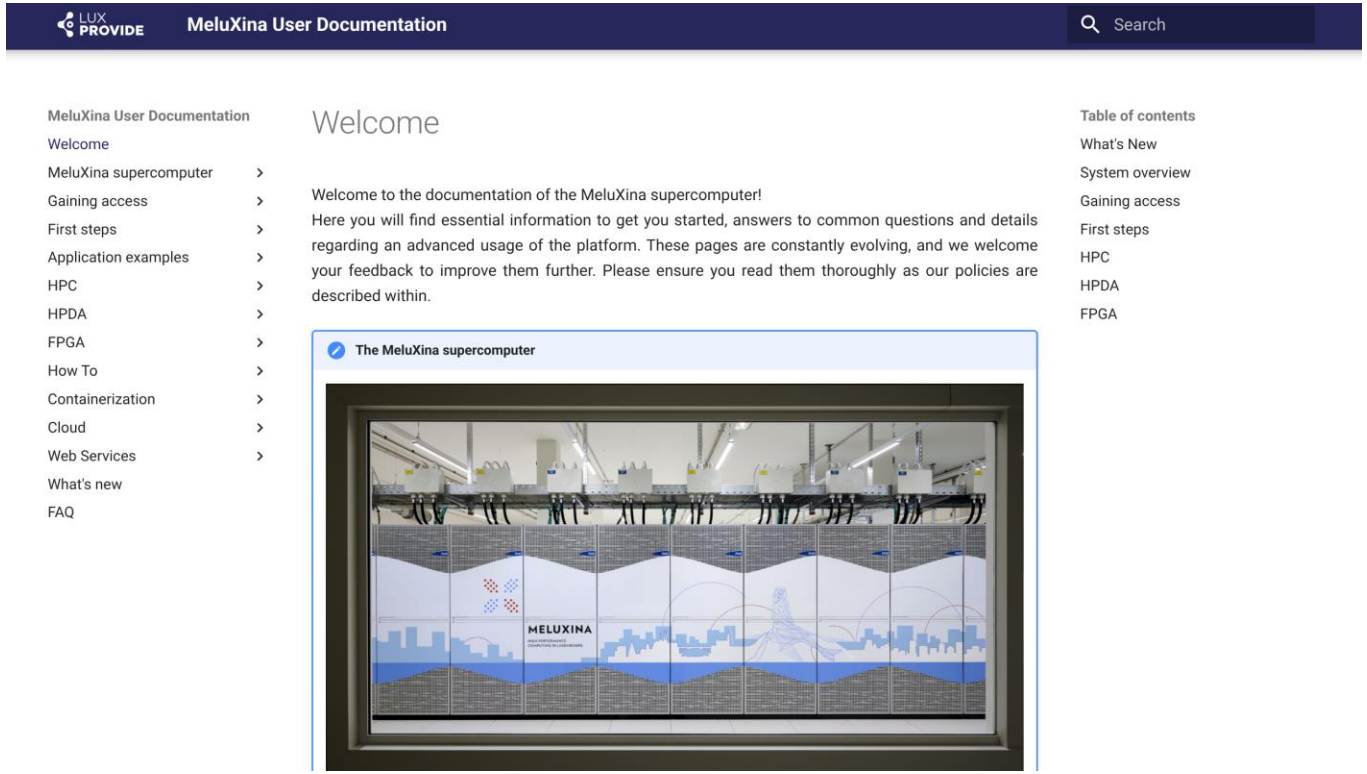
1. *Connect to the login node*
2. *Allocate resources:* $ **salloc –A p000000 –p cpu –qos default –t 20:00**
3. *Load Apptainer:* $ **module load Apptainer**
4. *Pull Docker images:* $ **apptainer pull docker://python:3.9.7-slim-bullseye**
5. *Run Apptainer images:* $ **apptainer run image.sif** (add --nvccli for accessing GPUs)

How to use Apptainer on MeluXina
https://docs.lxp.lu/containerization/apptainer/

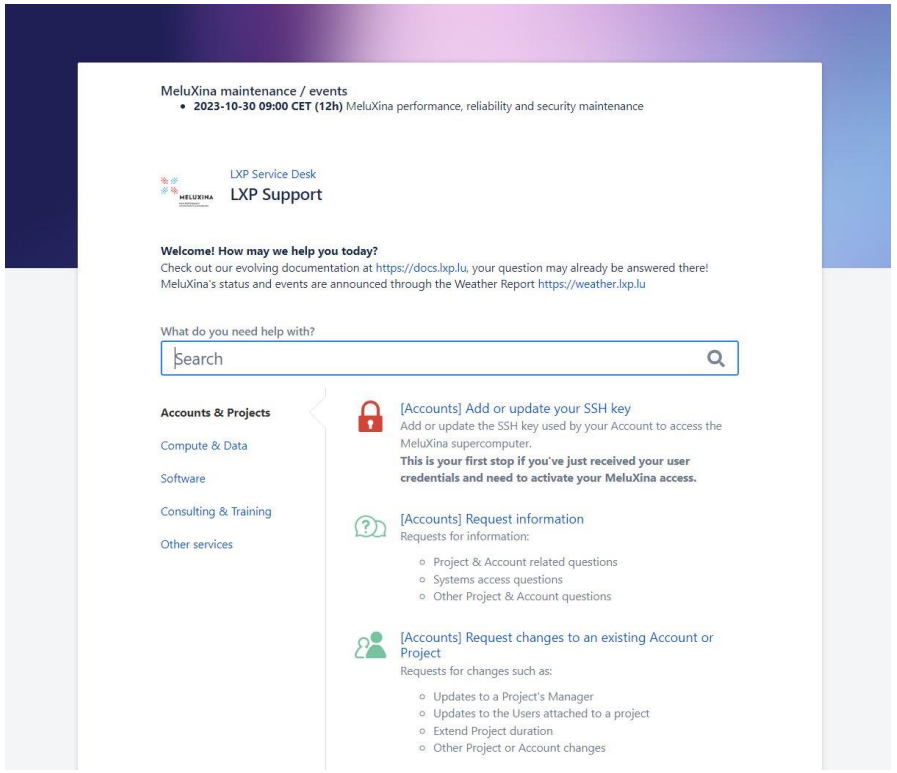
MeluXina: Getting help

Getting help

MeluXina User Documentation
https://docs.lxp.lu/



MeluXina User Support
https://servicedesk.lxp.lu




LUX
PROVIDE
EXPLORING
THE FRONTIERS
OF DIGITAL INTELLIGENCE
MADE IN
LUXEMBOURG